Boolean Matching Reversible Circuits: Algorithm and Complexity

Tian-Fu Chen[1,4,5] and Jie-Hong R. Jiang[1,2,3,4,5]
[1]*Graduate School of Advanced Technology, National Taiwan University*, Taipei, Taiwan
[2]*Graduate Institute of Electronics Engineering, National Taiwan University*, Taipei, Taiwan
[3]*Department of Electrical Engineering, National Taiwan University*, Taipei, Taiwan
[4]*Center for Quantum Science and Engineering, National Taiwan University*, Taipei, Taiwan
[5]*Physics Division, National Center for Theoretical Sciences*, Taipei, Taiwan
{d11k42001, jhjiang}@ntu.edu.tw

ABSTRACT

Boolean matching is an important problem in logic synthesis and verification. Despite being well-studied for conventional Boolean circuits, its treatment for reversible logic circuits remains largely, if not completely, missing. This work provides the first such study. Given two (black-box) reversible logic circuits that are promised to be matchable, we check their equivalences under various input/output negation and permutation conditions subject to the availability/unavailability of their inverse circuits. Notably, among other results, we show that the equivalence up to input negation and permutation is solvable in quantum polynomial time, while its classical complexity is exponential. This result is arguably the first demonstration of quantum exponential speedup in solving design automation problems. Also, as a negative result, we show that the equivalence up to both input and output negations is not solvable in quantum polynomial time unless UNIQUE-SAT is, which is unlikely. This work paves the theoretical foundation of Boolean matching reversible circuits for potential applications, e.g., in quantum circuit synthesis.

KEYWORDS

reversible circuits, Boolean matching, swap test, quantum algorithms

1 INTRODUCTION

Reversible logic circuits, simply called *reversible circuits*, are a class of Boolean circuits with n inputs and n outputs whose functions $\mathbb{B}^n \to \mathbb{B}^n$ form a bijection (one-to-one and onto) mapping, i.e., a permutation. Fundamentally, logical reversibility prevents information erasure and is a necessary condition for computation with extremely low energy dissipation [2, 9]. Remarkably, quantum computing [11] is intrinsically reversible. Every quantum gate, i.e., unitary operator, is reversible.[1] Reversible circuit synthesis has been extensively studied, e.g., [10, 12, 14–16], largely motivated by quantum computation due to the fact that several important quantum algorithms, such as Grover's algorithm [5], Simon's algorithm [13], etc., have oracle circuits, which are reversible circuits, as a key building block. In addition, reversible circuits also have applications in signal processing, cryptography, computer graphics, and nanotechnologies [12]. Constructing optimal reversible circuits is of practical importance. In this work, we are concerned with *Boolean matching* of reversible circuits.

[1]Although every quantum circuit is reversible, not every quantum circuit is referred to as a reversible circuit unless its unitary operator is a permutation matrix.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
DAC '24, June 23–27, 2024, San Francisco, CA, USA
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0601-1/24/06…$15.00
https://doi.org/10.1145/3649329.3657312

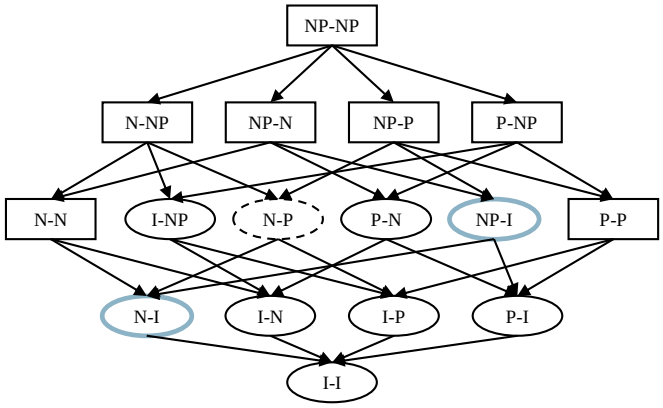

Figure 1: Domination relation among and computational complexities of various Boolean matching equivalences.

Conventionally, Boolean matching checks whether two (irreversible) logic circuits are equivalent up to the negation and/or permutation of inputs and/or outputs [6, 8]. It plays a vital role in logic synthesis and verification tasks such as technology mapping [1], engineering change order (ECO) [7], etc. Although well-studied for conventional Boolean circuits, Boolean matching on reversible circuits is largely, if not completely, missing. This work provides the first comprehensive study of Boolean matching reversible circuits and paves the theoretical foundation for potential applications. For example, template-based reversible logic synthesis [10] will greatly benefit from Boolean matching (extending from structural to functional matching), and so will oracle circuit synthesis and verification for quantum computation.

Given two (black-box) reversible logic circuits, also referred to as *oracles*, that are promised to be matchable, we explore the algorithm and complexity for Boolean matching on reversible circuits under different equivalences up to negation and/or permutation on their inputs and/or outputs. We define "X-Y equivalence" for X, Y ∈ {I, N, P, NP}, where X and Y denote equivalence conditions on the input and output sides, respectively, of the circuits, and I, N, P, and NP stand for identity, negation equivalence, permutation equivalence, and negation plus permutation equivalence, respectively. In total, there can be 16 combinations of equivalences, whose dominance relations are shown in the graph of Figure 1, where an edge (A, B) from A to B indicates equivalence B is subsumed by A. Note that the I-I equivalence is just the typical combinational equivalence problem without needing to identify negation and permutation conditions.

This work contributes to the first comprehensive characterization of the computational complexity in terms of the number of oracle access required to compute the negation/permutation conditions for the 16 equivalences. Figure 1 summarizes the results. The equivalences in ovals and rectangles correspond to easy (solvable in classical or quantum polynomial time) and hard (no easier than UNIQUE-SAT [4]) cases, respectively. As positive results, we provide concrete polynomial-time

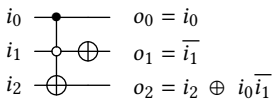

Figure 2: A reversible circuit example.

classical or quantum algorithms for checking the tractable equivalences. We note that the two gray-blue colored ovals (for N-I and NP-I equivalences) indicate their quantum, but not classical, polynomial-time solvability. To solve N-I and NP-I equivalences, we develop a quantum algorithm utilizing the *swap test* [3] as a subroutine.[2] The new algorithm achieves an exponential speedup over classical computation, adding to the rare quantum algorithm examples of attaining exponential speedup. Arguably, it is the first quantum algorithm with an exponential speedup in solving design automation and quantum program compilation problems. On the other hand, the dashed oval (for N-P equivalence) in Figure 1 indicates conditional classical polynomial-time solvability but with unknown quantum complexity. As negative results, we prove that N-N and P-P equivalences cannot be solved in quantum polynomial time unless UNIQUE-SAT can. Consequently, all other equivalences that dominate N-N or P-P equivalences are UNIQUE-SAT hard.

The rest of this paper is organized as follows. Section 2 provides essential backgrounds on reversible circuits and quantum computation. Section 3 formulates the Boolean matching problem of reversible circuits. For the equivalences that are polynomially solvable, Section 4 presents the corresponding algorithms and analyzes their complexities. Section 5 studies the equivalences that we prove to be harder than UNIQUE-SAT. Finally, we conclude this work in Section 6.

2 PRELIMINARIES

2.1 Reversible Circuits

An n-bit reversible circuit contains n input bits and n output bits (depicted with n lines with inputs at the left ends and outputs at the right ends) and implements a one-to-one and onto function $f : \mathbb{B}^n \to \mathbb{B}^n$, i.e., a permutation mapping. Clearly, f is reversible as its inverse function f^{-1} always exists. A reversible circuit can be generally represented by a circuit composed of multiple-controlled Toffoli (MCT) gates [12]. An MCT gate has $k = 0, 1, 2, \ldots$ control bits, each of which can be of a positive (indicated with a solid dot) or negative polarity (indicated with an empty circle), and conditionally flips a target bit (indicated with sign $\oplus$) when and only when all the negative and positive control bits are of values 0 and 1, respectively. In the special case of $k = 0$ and 1, the MCT gate corresponds to the NOT- and CNOT-gate (for a positive control bit), respectively. Figure 2 shows an example.

2.2 Quantum Computation

A *quantum bit* (*qubit*) can be in a superposition state over states $|0\rangle$ and $|1\rangle$, generally specified, in the Dirac notation, by $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, where α and $\beta \in \mathbb{C}$ are referred to as the *probability amplitudes*, with $|\alpha|^2 + |\beta|^2 = 1$. Specifically, $|\alpha|^2$ and $|\beta|^2$ correspond to the probabilities for measuring the state $|\psi\rangle$ be in $|0\rangle$ and $|1\rangle$, respectively. A quantum system of n qubits can be specified by $|\psi\rangle = a_{0\ldots0}|0\ldots0\rangle + \ldots + a_{1\ldots1}|1\ldots1\rangle$, or alternatively written as a column vector $|\psi\rangle = [a_{0\ldots0}, \ldots, a_{1\ldots1}]^{\mathrm{T}}$. The evolution of a quantum system of n qubits is governed by a sequence of quantum gates $U_1, \ldots, U_k$, for each U_i being a $2^n \times 2^n$ unitary matrix (or operator) over complex numbers satisfying $U_i^\dagger = U_i^{-1}$, i.e., its conjugate transpose equals its inverse. The gate sequence constitutes a quantum circuit with a global unitary matrix U equal to the matrix product $U_k \cdots U_1$. Applying U on an initial quantum state $|\psi\rangle$ leads to the final state $|\psi'\rangle = U|\psi\rangle$. Given two arbitrary states $|\psi_1\rangle$ and $|\psi_2\rangle$, a unitary matrix U preserves their inner product, i.e., $\langle\psi_1|U^\dagger U|\psi_2\rangle = \langle\psi_1|\psi_2\rangle$, where $\langle\psi_1|\psi_2\rangle$ denotes the inner product of $|\psi_1\rangle$ and $|\psi_2\rangle$, and $\langle\psi_1|U^\dagger U|\psi_2\rangle$ is the inner product of $U|\psi_1\rangle$ and $U|\psi_2\rangle$. Note that the function mapping defined by a reversible circuit can be represented as a permutation matrix, which is a unitary operator.

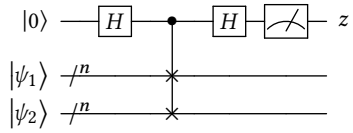

Figure 3: The circuit of quantum swap test.

In this work, we utilize the *swap test* [3], a quantum computation technique used to check how much two quantum states differ, in some of our algorithms. Given two n-qubit quantum states $|\psi_1\rangle$ and $|\psi_2\rangle$, the swap test applies the circuit shown in Figure 3 and measures the first qubit. After measurement, z is either in $|0\rangle$ of probability $\frac{1}{2} + \frac{1}{2}|\langle\psi_1|\psi_2\rangle|^2$ or in $|1\rangle$ of probability $\frac{1}{2} - \frac{1}{2}|\langle\psi_1|\psi_2\rangle|^2$. Hence, if $|\psi_1\rangle$ and $|\psi_2\rangle$ are identical, i.e., $|\langle\psi_1|\psi_2\rangle| = 1$, then the outcome z is always in $|0\rangle$. At the other extreme, if $|\psi_1\rangle$ and $|\psi_2\rangle$ are orthogonal, i.e, $|\langle\psi_1|\psi_2\rangle| = 0$, then the outcome z is in $|0\rangle$ and $|1\rangle$ with equal probability.

3 PROBLEM FORMULATION

The Boolean matching problem for reversible circuits can be stated as follows.

Problem 1 (Boolean matching reversible circuits). *Given two n-bit black-box reversible circuits C_1 and C_2 with C_1 and C_2 being promised to be X-Y equivalent for X, Y $\in$ {I, N, P, NP}, we are asked to find the corresponding negation functions $v_x, v_y : \{1, \ldots, n\} \to \mathbb{B}$ and permutation functions $\pi_x, \pi_y : \{1, \ldots, n\} \to \{1, \ldots, n\}$ of X and Y that make C_1 and C_2 X-Y equivalent, where $v(i) = 1$ indicates the i^{th} bit being negated and $\pi(i) = j$ indicates the i^{th} bit being permuted to the j^{th} bit.*

In the sequel, we omit the subscripts x, y in v and π when the input/output information is clear from the context or immaterial to the discussion. We denote the reversible circuits of functions v and π as C_v and C_π, respectively. Furthermore, we do not distinguish a reversible circuit C from its underlying unitary matrix. Hence, concatenating a reversible circuit C_A (on the left) followed by circuit C_B (on the right) is represented as the matrix product $C_B C_A$. For example, N-P equivalent C_1 and C_2 means there exist C_v and C_π to make $C_1 = C_\pi C_2 C_v$ for some input negation function v and output permutation function π.

Notice that Problem 1 is a promise problem; that is, the circuits under test are promised to be Boolean matchable under certain equivalences. Finding a solution to the promise problem is crucial to even answering the non-promise version of the problem. It is because as long as we can solve Problem 1, a solution of the negation and permutation conditions can be obtained even if the equivalence relation is not promised. With the found negation and permutation conditions, only a single round of equivalence checking is needed to validate the equivalence relation. In contrast, without knowing the negation and permutation conditions, one may need to try an exponential number of equivalence checking rounds for all v and π options.

Also note that Problem 1 assumes C_1 and C_2 are given as black boxes (oracles). The computational complexity of finding v and π functions is measured in terms of the number of queries to the oracles. A variant problem of Problem 1 is to relax the assumption given not only C_1 and

[2]Besides the swap-test-based algorithm, we develop two more algorithms inspired by Simon's algorithm [13] and have to omit them due to space limit.

C_2 but also their inverse circuits C_1^{-1} and C_2^{-1}. Surely, if C_1 and C_2 are given as white boxes, we can always derive their inverse circuits as they are reversible. In Section 4, we will discuss how this relaxation may possibly simplify the computation.

4 TRACTABLE EQUIVALENCES

In this section, we investigate Boolean matching equivalences {I-NP, N-P, P-N, NP-I, N-I, I-N, I-P, P-I} that permit polynomial time identification of negation and permutation functions. For these equivalences, we develop efficient algorithms and analyze their complexities as summarized in Table 1, where n is the bit size of the circuits and ϵ is the failure probability for randomized algorithms.

4.1 I-N Equivalence

Proposition 1. *For I-N equivalence, finding ν for $C_1 = C_\nu C_2$ is of $O(1)$ query complexity.*

We set all inputs of C_1 and C_2 to 0 and bit-wisely compare their outputs. Then, for $i = 1, \ldots, n$, we have $\nu(i) = 1$ if and only if the output patterns of C_1 and C_2 differ at the i^{th} bit. The computation requires only one oracle query.

4.2 I-P Equivalence

Proposition 2. *For I-P equivalence, finding π for $C_1 = C_\pi C_2$ is of $O(\log(n))$ query complexity if C_1^{-1} or C_2^{-1} is available, and of query complexity $O(\log(n)+\log(1/\epsilon))$ by a randomized algorithm with success probability $1-\epsilon$ if C_1^{-1} and C_2^{-1} are unavailable.*

If C_2^{-1} is available, let $C = C_1 C_2^{-1}$ by concatenating C_2^{-1} and C_1, and thus C is functionally equivalent to C_π. We use $\lceil \log_2 n \rceil$ input patterns to decide π as follows. For the i^{th} input pattern, the input of the j^{th} bit of the circuit is the i^{th} least significant bit of the binary code of index j. In other words, the input sequence to the j^{th} bit of the circuit is the binary code of j with the least significant bit arriving first. Then $\pi(p) = q$ if and only if the sequential input of the p^{th} bit is the same as the sequential output of the q^{th} bit. Hence, $O(\log n)$ oracle queries of C are needed. Similarly, if C_1^{-1} is available, we let $C = C_2 C_1^{-1}$, which equals $C_{\pi^{-1}}$. The same way applies to find π^{-1}, and thus π.

If both C_1^{-1} and C_2^{-1} are unavailable, the following randomized algorithm is applied to find π in two steps. First, repeat k times to feed random input patterns to both C_1 and C_2 and record their output patterns, where k is a number related to the failure probability to be explained shortly. Second, for each output bit $b_1 = 1, \ldots, n$ of C_1, find the unique output bit b_2 of C_2 such that they share the same output sequence, which indicates $\pi(b_1) = b_2$.

Table 1: Complexity of computing various equivalences.

Inverse circuit	Equivalence type	Computing paradigm	Complexity
available	N-I*, I-N*	classical	$O(1)$
	I-P*, P-I*, N-P**, P-N*, I-NP*, NP-I*	classical	$O(\log n)$
not available	I-N	classical	$O(1)$
	I-P, I-NP	classical	$O(\log n + \log(1/\epsilon))$
	P-I, P-N	classical	$O(n)$
	N-I	quantum	$O(n \log(1/\epsilon))$
	NP-I	quantum	$O(n^2 \log(1/\epsilon))$

"*" indicates one inverse circuit of either C_1 or C_2 is required.
"**" indicates both inverse circuits of C_1 and C_2 are needed.

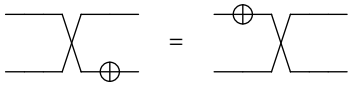

Figure 4: Exchanging the order between negation and permutation circuits.

To ensure the second step finds a unique b_2 for each b_1, there cannot be two bits sharing the same output sequence. Hence, we require k iterations of the first step to make the output sequence long enough. Note that the range under the mapping of a reversible circuit must cover all 2^n combinations, and thus, different input patterns must yield different output patterns. As long as the input patterns are uniformly generated at random, the output patterns are also uniformly at random. Let $B_1(j)$ be the output sequence of the j^{th} bit in C_1. Since there are 2^k possible output sequences for any output bit j, the success probability Pr that $B_1(j_1) \neq B_1(j_2)\ \forall j_1 \neq j_2$ is

$$\begin{aligned} \Pr &= \frac{(2^k)(2^k-1)\ldots(2^k-n+1)}{(2^k)^n} \geq \left(\frac{2^k-n+1}{2^k}\right)^n \\ &\approx 1 - \frac{n(n-1)}{2^k} \quad (\text{when } n \ll 2^k). \end{aligned} \tag{1}$$

To make $\Pr \geq 1-\epsilon$, we need $k \geq \log_2 \frac{n(n-1)}{\epsilon} = O(\log(n) + \log(1/\epsilon))$. Therefore, the query complexity of the algorithm is $O(\log(n)+\log(1/\epsilon))$.

4.3 I-NP Equivalence

Proposition 3. *For I-NP equivalence, finding ν and π for $C_1 = C_\pi C_\nu C_2$ is of $O(\log(n))$ query complexity if either C_1^{-1} or C_2^{-1} is available, and of complexity $O(\log(n)+\log(1/\epsilon))$ by a randomized algorithm with success probability $1-\epsilon$ if C_1^{-1} and C_2^{-1} are unavailable.*

Observe that the order of C_ν and C_π can be exchanged according to the identity shown in Figure 4. If C_2^{-1} is available, let $C = C_1 C_2^{-1}$ by concatenating C_2^{-1} and C_1, which is functionally equivalent to $C_\pi C_\nu$. We temporally let $C_\pi C_\nu = C_{\nu'} C_{\pi'}$. To decide ν', we set all inputs to 0. Since all inputs have the same value, the initial permutation circuit $C_{\pi'}$ has no effect, and thus, $\nu'(i) = 1$ if and only if the output of the i^{th} bit is 1. This step requires only one oracle query. After $\nu'(i)$ is decided, the method mentioned in Section 4.2 can be slightly modified to find π', which needs $O(\log(n))$ oracle queries. Finally, ν' and π' can be transformed back to get ν and π. Similarly, if C_1^{-1} is available, we let $C = C_2 C_1^{-1}$ and follow a similar procedure to obtain ν^{-1} and π^{-1}, and transform them back to ν and π.

If both C_1^{-1} and C_2^{-1} are unavailable, a randomized algorithm similar to the one in Section 4.2 can be employed to find π. The modification is that when we look for the unique b_2 in C_2 for each bit b_1 in C_1, we now recognize two kinds of b_2. One is the same as before, where the b_1^{th} bit in C_1 has the same output sequence as the b_2^{th} bit in C_2, which indicates $\pi(b_1) = b_2$ and $\nu(b_2) = 0$. The other is that the b_1^{th} bit in C_1 has exactly the bitwise-flipped output sequence to the b_2^{th} bit in C_2, which indicates $\pi(b_1) = b_2$ and $\nu(b_2) = 1$. An analysis similar to that in Section 4.2 gives the $O(\log(n) + \log(1/\epsilon))$ complexity.

4.4 P-I Equivalence

Proposition 4. *For P-I equivalence, finding π for $C_1 = C_2 C_\pi$ is of $O(\log(n))$ query complexity if C_1^{-1} or C_2^{-1} is available, and of $O(n)$ complexity if C_1^{-1} and C_2^{-1} are unavailable.*

If C_1^{-1} (resp. C_2^{-1}) is available, we obtain $C = C_1^{-1}C_2$ (resp. $C = C_1C_2^{-1}$), which is functionally equivalent to $C_{\pi^{-1}}$ (resp. C_π). As proposed in Section 4.2, π can be decided in $O(\log(n))$ number of oracle queries.

If both C_1^{-1} and C_2^{-1} are unavailable, π can be found in $O(n)$ number of oracle queries with the following algorithm in two steps. First, prepare n one-hot input patterns. The i^{th} pattern assigns 0 to all bits except one 1 to the i^{th} bit. For the i^{th} input pattern, collect the corresponding output patterns $P_{o1,i}$ of C_1 and $P_{o2,i}$ of C_2. Let $M_1[P_{o1,i}] = i$ and $M_2[i] = P_{o2,i}$. Second, for each $i = 1, \ldots, n$, let $\pi(i) = M_1[M_2[i]]$. Thereby, π is found in $O(n)$ oracle queries.

4.5 N-I Equivalence

Proposition 5. *For N-I equivalence, finding ν for $C_1 = C_2C_\nu$ is of $O(\log(n))$ query complexity if either C_1^{-1} or C_2^{-1} is available, and of $O(n\log(1/\epsilon))$ query complexity by a randomized quantum algorithm with success probability $1-\epsilon$ if C_1^{-1} and C_2^{-1} are unavailable but C_1 and C_2 can take quantum states as inputs.*

If C_2^{-1} is available, let $C = C_2^{-1}C_1 = C_\nu$. By setting all inputs of C to 0, $\nu(i) = 1$ if and only if the i^{th} bit at output is 1. Similarly, if C_1^{-1} is available, let $C = C_1^{-1}C_2 = C_{\nu^{-1}}$. Observing $C_{\nu^{-1}} = C_\nu$, the same approach can be applied to decide ν.

If C_1^{-1} and C_2^{-1} are both unavailable, Theorem 1 provides the complexity lower bound of classical algorithms.

Theorem 1. *If C_1^{-1} and C_2^{-1} are unavailable, a classical algorithm to find the negation function for N-I equivalence needs at least $\Omega(2^{n/2})$ oracle queries.*

Proof. Given black boxes C_1 and C_2, they can be accessed only by queries, specifically, one output response per input query. Only when C_1 and C_2 yield the same output upon two input patterns to C_1 and C_2, we can infer ν by comparing these two input patterns. A classical algorithm can only randomly try different input patterns until C_1 and C_2 produce the same output pattern, i.e., a collision happens. Assuming k queries to C_2 are made to match an output pattern of C_1, then the probability of collisions not happening is

$$\begin{aligned} \Pr &= \frac{(2^n)(2^n-1)\ldots(2^n-k+1)}{(2^n)^k} \geq \left(\frac{2^n-k+1}{2^n}\right)^k \\ &\approx 1-\frac{k(k-1)}{2^n} \quad (\text{when } k \ll 2^n). \end{aligned} \tag{2}$$

For Pr being smaller than some constant threshold c, the query number k should be at least $\sqrt{(1-c)\cdot 2^n}$, which yields the lower bound $\Omega(2^{n/2})$. □

Fortunately, assuming that the reversible circuits C_1 and C_2 can take quantum states as input, finding ν for N-I equivalence without C_1^{-1} and C_2^{-1} is solvable in quantum polynomial time by Algorithm 1. The key idea is to utilize superposition states to disable NOT-gates, such that only one NOT-gate is active at a time. In addition, the swap test [3] is exploited as a subroutine to compare two quantum states. Algorithm 1 proceeds in n iterations, where the i^{th} iteration decides the value of $\nu(i)$. In the i^{th} iteration, the inputs of the i^{th} qubit of both circuits are set to $|0\rangle$, while all other qubits are initialized to $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$ in lines 2 to 3. Line 4 sets $\nu(i) = 0$ in default. In lines 5 to 8, k iterations of swap tests are performed for k being determined by the failure probability as to be explained. In line 6, we execute C_1 and C_2 on input pattern $P_{in(i)}$ to obtain their final states $C_1(P_{in(i)})$ and $C_2(P_{in(i)})$, and these two final states are sent to the swap test. If the measurement outcome of the swap test is 1, we conclude that $\nu(i) = 1$, and the iteration is terminated, as shown in lines 7 to 8. Otherwise, if the measurement outcome is 0 for k times, then we have high confidence that $\nu(i) = 0$.

Algorithm 1: Quantum Algorithm for N-I Equivalence

```
Input  : C1, C2: Two n-bit reversible circuits
Output : ν: Negation function making C1 = C2Cν
1 for i = 1, 2, ..., n
2 |   P_in(i) ← [|+⟩, |+⟩, ..., |+⟩]
3 |   P_in(i)[i] ← |0⟩
4 |   ν(i) ← 0
5 |   for j = 1, 2, ..., k
6 |   |   if SWAP_TEST(C1(P_in(i)), C2(P_in(i))) = 1
7 |   |   |   ν(i) ← 1
8 |   |   |   break
9 return ν
```

The correctness of Algorithm 1 is established below. We first show that $\nu(1)$ can be correctly decided, and $\nu(i)$ for $i = 2, \ldots, n$ follow with the same derivation. When deciding $\nu(1)$, the input states of C_1 and C_2 are both prepared as $|\psi\rangle = |0\rangle \otimes |+\rangle \otimes \ldots \otimes |+\rangle$. The output states of C_1 and C_2 are $C_1|\psi\rangle$ and $C_2|\psi\rangle$, respectively. Since $C_1 = C_2C_\nu$, the output state of C_1 can also be written as $C_2C_\nu|\psi\rangle$. Let $C_\nu|\psi\rangle = |\psi'\rangle$. Note that a NOT-gate is a Pauli X operator, whose application on $|+\rangle$ has no effect as $X|+\rangle = |+\rangle$. Hence, only the NOT-gate, if it exists, at the first bit in C_ν can make an effect on $|\psi\rangle$. There are two cases: In the first case, the first bit in C_ν has a NOT-gate, i.e., $\nu(1) = 1$. Then $|\psi'\rangle = |1\rangle \otimes |+\rangle \otimes \ldots \otimes |+\rangle$, and $\langle\psi'|\psi\rangle = \langle 1|0\rangle\langle +|+\rangle \cdots \langle +|+\rangle = 0 \cdot 1 \cdots 1 = 0$. Note that the final state of C_1 and C_2 are $C_2|\psi'\rangle$ and $C_2|\psi\rangle$, respectively. As mentioned in Section 2, quantum circuits must preserve the inner product of states. Therefore, after applying C_2 on both $|\psi'\rangle$ and $|\psi\rangle$, the final states of the two circuits still have inner product 0. Finally, when applying the swap test on the output states of C_1 and C_2, there is a 50% probability of obtaining a measurement outcome 1.

In the second case, the first bit in C_N does not have a NOT-gate, i.e., $\nu(1) = 0$. In this case, the output states of C_1 and C_2 are identical. When we apply the swap test to them, we always obtain a measurement outcome of 0. Hence, once the measurement outcome is 1, we can conclude $\nu(1) = 1$. Otherwise, if the measurement outcome is 0 for k times, $\nu(1) = 0$ is of high confidence. The probability of success is $1 - 1/2^k$. To make the failure probability $\leq \epsilon$, we require $k \geq \log_2(1/\epsilon)$. Since a similar process is used to decide $\nu(i)$ for all $i = 1, \ldots, n$, the query complexity of Algorithm 1 is $O(n\log(1/\epsilon))$.

4.6 NP-I Equivalence

Proposition 6. *For NP-I equivalence, finding ν and π for $C_1 = C_2C_\pi C_\nu$ is of $O(\log(n))$ query complexity if C_1^{-1} or C_2^{-1} is available, and of $O(n^2\log(1/\epsilon))$ query complexity by a randomized quantum algorithm with success probability $1-\epsilon$ if C_1^{-1} and C_2^{-1} are unavailable but C_1 and C_2 can take quantum states as inputs.*

If C_1^{-1} or C_2^{-1} is available, a method similar to that in Section 4.3 can decide ν and π in $O(\log(n))$ oracle queries. If C_1^{-1} and C_2^{-1} are both unavailable, the following quantum algorithm can be applied. First, find C_π by deciding whether the b_1^{th} bit in C_1 is mapped to the b_2^{th} bit in C_2 for all $b_1, b_2 \in \{1, 2, \ldots, n\}$. For each (b_1, b_2) pair, initialize the b_1^{th} qubit in C_1 and the b_2^{th} qubit in C_2 to $|-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$, while all other qubits are initialized to $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$. Then, execute both circuits and compare their final states by the swap test. Let $\pi(b_2) = b_1$ if and only if the measurement outcome of the swap test is 0 for k times. After π is found, Algorithm 1 can be slightly modified to find ν further.

The key idea of the algorithm is to disable C_ν first, so as to focus on finding C_π. To disable C_ν, choose the input state to be $|+\rangle$ and

$|-\rangle$. Note that a NOT-gate, i.e., a Pauli X operator, acting on $|-\rangle$ yields $X|-\rangle = -|-\rangle$, where the global phase -1 can be ignored. Moreover, the input states of C_1 and C_2 are identical when $b_1 = b_2$ and are orthogonal when $b_1 \neq b_2$. Therefore, the final states of C_1 and C_2 are identical if $\pi(b_2) = b_1$, and orthogonal otherwise. By applying the swap test k times on the final states of C_1 and C_2, we can decide whether $\pi(b_2) = b_1$. Again, we can derive that $k = \log_2(1/\epsilon)$ to make the failure probability $\leq \epsilon$, so the overall complexity of the algorithm is $O(n^2 \log(1/\epsilon))$.

4.7 P-N Equivalence

Proposition 7. *For P-N equivalence, finding π and ν for $C_1 = C_\nu C_2 C_\pi$ is of $O(\log(n))$ query complexity if C_1^{-1} or C_2^{-1} is available, and of $O(n)$ query complexity if C_1^{-1} and C_2^{-1} are unavailable.*

The result comes from the fact that this problem can be reduced to P-I equivalence testing, so it has the same complexity as P-I equivalence. Regardless of whether C_1^{-1} and C_2^{-1} are available, we first decide ν by setting all inputs to 0 and observing the outputs. Since all inputs are the same, input permutation makes no effect. Then $\nu(i) = 1$ if and only if the output patterns of C_1 and C_2 differ at the i^{th} bit. This step only takes $O(1)$ oracle queries and does not affect the complexity analysis. After ν is found, we obtain $C_3 = C_\nu C_2$. Note that C_1 and C_3 are P-I equivalent now, and the P-N equivalence can be reduced to P-I equivalence.

4.8 N-P Equivalence

Proposition 8. *For N-P equivalence, finding ν and π for $C_1 = C_\pi C_2 C_\nu$ is of $O(\log(n))$ query complexity if C_1^{-1} and C_2^{-1} are both available.*

If inverses of C_1 and C_2 are both available, the method mentioned in Section 4.7 is applicable to find ν and π by the fact that $C_1^{-1} = C_{\nu^{-1}} C_2^{-1} C_{\pi^{-1}}$. Then ν^{-1} and π^{-1} are transformed to ν and π. The complexity is the same as P-N equivalence testing.

If ν and π when both C_1^{-1} and C_2^{-1} are unavailable, whether there exists an efficient quantum algorithm remains an open problem for our future work.

5 INTRACTABLE EQUIVALENCES

We show that the equivalence classes not solved in Section 4 are more difficult than the promise *UNIQUE-SAT problem* [4]. Given a conjunctive normal form (CNF) Boolean formula φ that is promised to have at most one satisfying assignment, the UNIQUE-SAT problem asks to decide whether it is satisfiable. It has been proved that SAT is randomly reducible to UNIQUE-SAT [17], and thus UNIQUE-SAT is believed to be a difficult problem. In what follows, we will show that the UNIQUE-SAT problem is polynomially reducible to N-N and P-P equivalences. Since equivalences {NP-NP, N-NP, NP-N, NP-P, P-NP} subsume N-N or P-P, they are thus harder than UNIQUE-SAT.

5.1 Hardness of N-N Equivalence

Theorem 2. *For N-N equivalence, finding ν_x and ν_y for $C_1 = C_{\nu_y} C_2 C_{\nu_x}$ is no easier than UNIQUE-SAT.*

Proof. We establish a polynomial-time reduction from UNIQUE-SAT to N-N equivalence as follows. Consider a CNF formula $\varphi = c_1 \wedge c_2 \wedge \ldots \wedge c_m$ over variables $\{x_1, x_2, \ldots, x_n\}$ promised to have at most one satisfying assignment. Let clause $c_i = (\ell_{i,1} \vee \ell_{i,2} \vee \ldots \vee \ell_{i,k_i})$, a disjunction of k_i literals. We construct the UNIQUE-SAT encoding reversible circuit C_1 as shown in the black part of Figure 5(a) (excluding the red part), where the first n bits $b_{x_1}, \ldots, b_{x_n}$ correspond to the input variables, the next m ancilla bits $b_{a_1}, \ldots, b_{a_m}$ correspond to the valuations of the clauses, and two extra ancilla bits b_b and b_z are used to generate the value of φ. Each $U(\varphi) = U(c_m) \cdots U(c_1)$ is the concatenation of the clause-encoding circuits. For a clause $c_i = \ell_{i,1} \vee \ldots \vee \ell_{i,k_i}$, its clause-encoding circuit is an MCT gate followed by a NOT-gate. In the MCT gate, each literal $\ell_{i,j}$ in c_i corresponds to a control bit. If $\ell_{i,j}$ equals x_s (resp. $\overline{x_s}$), then a negative (resp. positive) control bit is applied on b_{x_s}. The target bit of the MCT gate and the NOT-gate are applied on b_{a_i}. E.g., the clause-encoding circuit of clause $c_1 = \overline{x_1} \vee x2 \vee \overline{x_3}$ is shown in Figure 5(b). Hence, for the whole $U(\varphi)$ block, if the input value of b_{x_j} equals x_j and the input value of b_{a_i} equals a_i, then the output values of all b_{x_j} remain the same as their input values, while the output value of b_{a_i} equals $a_i \oplus (\overline{\ell_{i,1}} \ldots \overline{\ell_{i,k_i}}) \oplus 1 = a_i \oplus c_i$. Note that $U(\varphi)^{-1} = U(\varphi)$.

For the overall UNIQUE-SAT encoding circuit in Figure 5(a), let v denote the input value of bit b_v. We note that the value of b_{a_i} is changed to $a_i \oplus c_i$ at time t_2 and t_4 and is returned back to a_i at time t_2 and the end of the circuit. Moreover, the value of b_b is changed to $b \oplus (\overline{a_1} \ldots \overline{a_m})$ at time t_1 and returned back to b at time t_3. It can be derived that the output values of the first $n+m+1$ bits are the same as their input values, while the output value of b_z is $z \oplus f$, for

$$\begin{aligned} f &= \left[\bigwedge_{i=1}^{m} (a_i \oplus c_i)(b \oplus (\overline{a_1} \ldots \overline{a_m}))\right] \oplus \left[\bigwedge_{i=1}^{m} (a_i \oplus c_i) b\right] \\ &= [(a_1 \oplus c_1) \ldots (a_m \oplus c_m)] \wedge (\overline{a_1} \ldots \overline{a_m}) \\ &= (c_1 \ldots c_m) \wedge (\overline{a_1} \ldots \overline{a_m}) \quad = \varphi \wedge (\overline{a_1} \ldots \overline{a_m}). \end{aligned} \tag{3}$$

Hence, $f = 1$ if and only if all a_i's are 0 and $(x_1, \ldots, x_n)$ is a satisfying assignment of φ.

On the other hand, we can construct a circuit C_2 shown in Figure 5(c). Let v denote the input value of bit b_v. It is easy to derive that the output values of the first $n + m + 1$ bits are the same as their input values, while the output value of b_z is $z \oplus g$, where $g = (x_1 \ldots x_n) \wedge (\overline{a_1} \ldots \overline{a_m})$. Taking the UNIQUE-SAT encoding circuit as C_1 and comparing f and g, it can be verified that C_1 and C_2 are N-N equivalent if and only if φ is satisfiable. The intuition is as follows. First, $\pi_x(i) = \pi_y(i)$ for $i = 1, \ldots, n + m + 1$ must hold to ensure the output values of the first $n + m + 1$ bits are the same as their input values. Then we observe that two NOT-gates applied before and after a control bit will flip the control polarity. Therefore, if $v_1(i) = v_2(i) = 1$, then x_i is in fact negatively controlling, indicating that $x_i = 0$ is the necessary condition to make $\varphi = 1$. Otherwise, $v_1(i) = v_2(i) = 0$ indicates that x_i is positively controlling, so $x_i = 1$ is the necessary condition to make $\varphi = 1$. Therefore, once we can find ν_x and ν_y to make C_1 and C_2 N-N equivalent, the unique satisfying assignment of φ is found. Even if we do not know whether $C_1 = C_{\nu_y} C_2 C_{\nu_x}$, we can still try the process and obtain a candidate solution. The validity of the candidate solution can be easily verified in linear time by substituting it into φ. Moreover, the reduction process is polynomial since there are only $8m + 4$ MCT gates in the UNIQUE-SAT encoding circuit. Hence, UNIQUE-SAT is polynomially reducible to the N-N equivalence problem, and the theorem follows. □

5.2 Hardness of P-P Equivalence

Theorem 3. *For P-P equivalence, finding π_x and π_y for $C_1 = C_{\pi_y} C_2 C_{\pi_x}$ is no easier than UNIQUE-SAT.*

Proof. Consider again the CNF formula φ in Section 5.1. We create another CNF formula φ' by adding n extra variables $y_1, \ldots, y_n$ and setting $\varphi' = \varphi \wedge c_{m+1} \wedge \ldots \wedge c_{m+2n}$, where $(c_{m+2j-1}) \wedge (c_{m+2j}) = (x_j \vee y_j) \wedge (\overline{x_j} \vee \overline{y_j})$, $j = 1, \ldots n$. That is, we make a dual-rail encoding on the variables of φ in φ' and set $y_j = \overline{x_j}$ for all y_j. Hence, φ is satisfiable if and only if φ' is satisfiable. Then the same method mentioned in *Section* 5.1 is applied to encode φ' into C_1, as shown in Figure 5(a) (including both the black and red parts). Again, the output values of the first $4n + m + 1$ bits are always the same as their input values. For the last bit b_z, if its input value is z, then its output value is $z \oplus f$, where

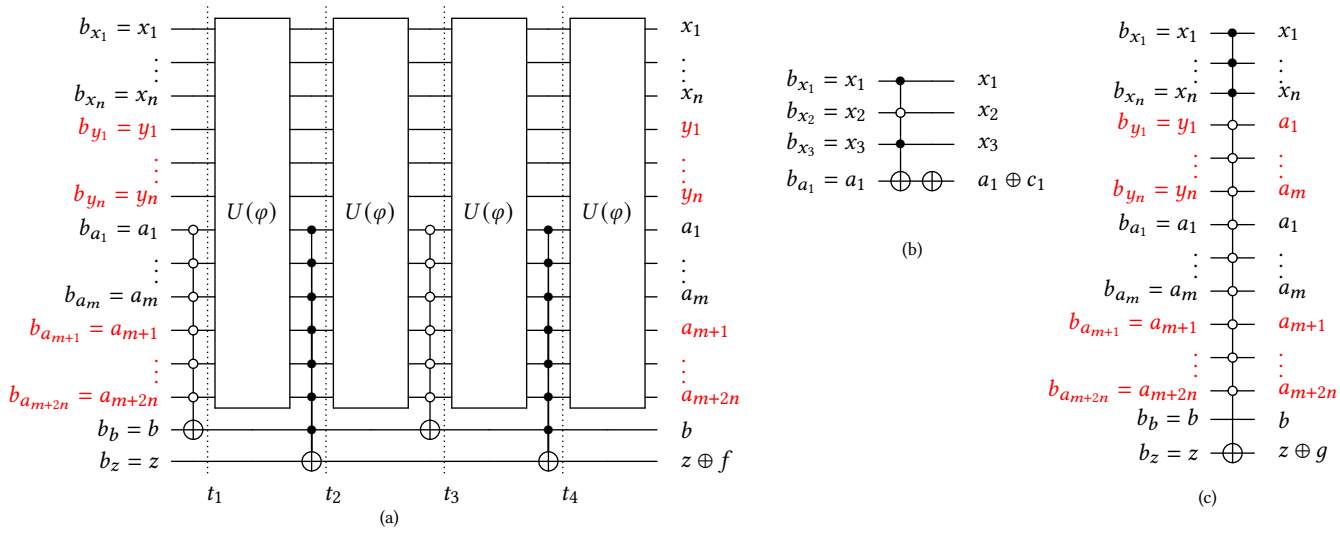

Figure 5: (a) The UNIQUE-SAT encoding circuit, (b) a clause-encoding circuit $U(c)$ of clause $c = \overline{x_1} \vee x2 \vee \overline{x_3}$, (c) the C_2 circuit for Boolean matching.

$f = \varphi' \wedge (\overline{a_1} \ldots \overline{a_{m+2n}})$. On the other hand, we can construct a circuit C_2 shown in Figure 5(c) (including both black and red parts), where the first n bits are positive-control bits, the $(n+1)^{\text{th}}$ to the $(4n+m)^{\text{th}}$ bits are negative-control bits, and the last bit b_z is the target bit.

By comparing f and g, it can be verified that C_1 and C_2 are P-P equivalent if and only if φ' is satisfiable. The intuition is as follows. First, we note that $\pi_x^{-1} = \pi_y$ must hold to ensure the output values of the first $4n + m + 1$ bits are the same as their input values. Therefore, the input and output permutations are equivalently just permuting the control bits. Second, the i^{th} bit is positively controlling if it falls in the positive-control region ($0 < \pi_x^{-1}(i) \leq n$), or it is negatively controlling if it falls in the negative-control region ($n < \pi_x^{-1}(i) \leq 4n + m$). If x_i is permuted to the positive-control region and y_i is permuted to the negative-control region, then $x_i = \overline{y_i} = 1$ is the necessary condition to make $\varphi' = 1$. Otherwise, if x_i is permuted to the negative-control region and y_i is permuted to the positive-control region, then $x_i = \overline{y_i} = 0$ is the necessary condition to make $\varphi' = 1$. Therefore, once we can find π_x and π_y to make C_1 and C_2 P-P equivalent, the unique satisfying assignment of φ' is found, which can be easily transformed to the unique satisfying assignment of φ. Even if we do not know whether C_1 and C_2 are P-P equivalent or not, we can still try the process and obtain a candidate solution. The validity of the candidate solution can be easily verified by substituting it into φ. Moreover, the reduction process is polynomial since there are only $8m + 4$ MCT gates in the UNIQUE-SAT encoding circuit. Hence, UNIQUE-SAT is polynomially reducible to the P-P equivalence problem, and the theorem follows. □

6 CONCLUSIONS AND FUTURE WORK

This work provided the first comprehensive study on various equivalences for Boolean matching of reversible circuits by characterizing their computational complexities. For the tractable equivalences, polynomial-time (classical or quantum) algorithms were devised. For the intractable equivalences, their hardness results were established. The foundation paved in this work may open new Boolean matching applications, e.g., in template-based reversible logic synthesis and in quantum program compilation for oracle circuit minimization. Moreover, our swap-test-based algorithm demonstrates the first example with an exponential quantum speedup over classical computation in design automation research. It may inspire the development of new types of quantum algorithms and applications. For future work, we intend to resolve the remaining open problem regarding the quantum complexity of N-P equivalence.

ACKNOWLEDGMENTS

This work was supported in part by the National Science and Technology Council of Taiwan under grants 112-2119-M-002-017 and 113-2119-M-002-024, and the NTU Center of Data Intelligence: Technologies, Applications, and Systems under grant NTU-113L900903. The authors thank IBM Q Hub at NTU and Quantum Technology Cloud Computing Center at NCKU for supporting experimental validation.

REFERENCES

[1] Luca Benini and Giovanni De Micheli. 1997. A survey of Boolean matching techniques for library binding. *ACM Transactions on Design Automation of Electronic Systems* 2, 3 (1997), 193–226.
[2] Charles H Bennett. 1973. Logical reversibility of computation. *IBM journal of Research and Development* 17, 6 (1973), 525–532.
[3] Harry Buhrman, Richard Cleve, John Watrous, and Ronald De Wolf. 2001. Quantum fingerprinting. *Physical Review Letters* 87, 16 (2001), 167902.
[4] Oded Goldreich. 2006. On promise problems: A survey. In *Theoretical Computer Science: Essays in Memory of Shimon Even*. Springer, 254–290.
[5] Lov K Grover. 1996. A fast quantum mechanical algorithm for database search. In *Proc. STOC*. 212–219.
[6] Hadi Katebi and Igor L. Markov. 2010. Large-scale Boolean matching. In *Proc. DATE*. 771–776.
[7] Smita Krishnaswamy, Haoxing Ren, Nilesh Modi, and Ruchir Puri. 2009. DeltaSyn: An efficient logic difference optimizer for ECO synthesis. In *Proc. ICCAD*. 789–796.
[8] Chih-Fan Lai, Jie-Hong R. Jiang, and Kuo-Hua Wang. 2010. BooM: A decision procedure for Boolean matching with abstraction and dynamic learning. In *Proc. DAC*. 499–504.
[9] Rolf Landauer. 1961. Irreversibility and heat generation in the computing process. *IBM journal of research and development* 5, 3 (1961), 183–191.
[10] D. Michael Miller, Dmitri Maslov, and Gerhard W. Dueck. 2003. A transformation based algorithm for reversible logic synthesis. In *Proc. DAC*. 318–323.
[11] M. A. Nielsen and I. L. Chuang. 2010. *Quantum Computation and Quantum Information*. Cambridge University Press.
[12] Mehdi Saeedi and Igor L Markov. 2013. Synthesis and optimization of reversible circuits—a survey. *ACM Computing Surveys* 45, 2 (2013), 1–34.
[13] Daniel R Simon. 1997. On the power of quantum computation. *SIAM Journal on Computing* 26, 5 (1997), 1474–1483.
[14] Mathias Soeken, Stefan Frehse, Robert Wille, and Rolf Drechsler. 2012. Revkit: A toolkit for reversible circuit design. *Journal of Multiple-Valued Logic and Soft Computing* 18, 1 (2012), 55–65.
[15] Mathias Soeken, Martin Roetteler, Nathan Wiebe, and Giovanni De Micheli. 2017. Hierarchical reversible logic synthesis using LUTs. In *Proc. DAC*. 1–6.
[16] Mathias Soeken, Robert Wille, Oliver Keszocze, D Michael Miller, and Rolf Drechsler. 2015. Embedding of large Boolean functions for reversible logic. *ACM Journal on Emerging Technologies in Computing Systems* 12, 4 (2015), 1–26.
[17] Leslie G Valiant and Vijay V Vazirani. 1985. NP is as easy as detecting unique solutions. In *Proc. STOC*. 458–463.

Voronoi Diagram-based Multiple Power/Ground Plane Generation on Redistribution Layers in 3D ICs

Chia-Wei Lin
College of Artificial Intelligence,
National Yang Ming Chiao Tung University
Tainan, Taiwan

Jing-Yao Weng
Institute of Pioneer Semiconductor Innovation,
National Yang Ming Chiao Tung University
Hsinchu, Taiwan

I-Te Lin
Synopsys, Inc.
Hsinchu, Taiwan

Ho-Chieh Hsu
Synopsys, Inc.
Hsinchu, Taiwan

Chia-Ming Liu
Synopsys, Inc.
Hsinchu, Taiwan

Mark Po-Hung Lin
Institute of Pioneer Semiconductor Innovation,
National Yang Ming Chiao Tung University
Hsinchu, Taiwan

ABSTRACT

In three-dimensional integrated circuits, the interconnection design among chiplets on redistribution layers (RDLs) is crucial for achieving high-performance computing systems. To optimize the inter-chip connections, most of the previous works focused on automatic signal net routing and pin assignment. The power/ground plane generation, is still a manual and time-consuming task, especially when generating the power planes of more than ten power supplies on a limited number of RDLs. This paper proposes a novel Voronoi diagram-based multiple power/ground plane generation methodology that simultaneously optimizes the power/ground planes of all power/ground nets by utilizing the white space of given RDLs, while considering the signal routing blockages, power integrity, and complex design rules. Experimental results show that the proposed approach can achieve not only optimal area utilization but also the best cross-layer power integrity in terms of the total number of redundant vias.

KEYWORDS

3D-IC, RDL, redistribution layer, routing, power plane, power integrity, redundant via

ACM Reference Format:
Chia-Wei Lin, Jing-Yao Weng, I-Te Lin, Ho-Chieh Hsu, Chia-Ming Liu, and Mark Po-Hung Lin. 2024. Voronoi Diagram-based Multiple Power/Ground Plane Generation on Redistribution Layers in 3D ICs. In *61st ACM/IEEE Design Automation Conference (DAC '24), June 23–27, 2024, San Francisco, CA, USA.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3649329.3657315

1 INTRODUCTION

In advanced packing technologies, or heterogeneous integration of three-dimensional integrated circuits (3D ICs), the interposer, which consists of several redistribution layers (RDLs), is the medium to deliver massive power supplies and a large number of signals among different chiplets. Compared with traditional printed circuit boards (PCBs), RDLs offer higher I/O density, shorter interconnections, and better electronic characteristics, resulting in high-performance computation [1, 2].

As the number of power supplies may be much more than the number of RDLs in modern 3D ICs, multiple power/ground nets usually share common RDLs together with other signal nets. Different from the mesh [3], tree [4], stripe [5], and strap [6] structures of the intra-chip power delivery network, the inter-chip power nets in 3D ICs usually form irregular areas, or power planes, on each RDL. When generating the power planes of each power net on different RDLs, it is essential to enlarge the power plane area and the number of redundant vias for reducing IR-drop, mitigating electromigration, and enhancing power integrity.

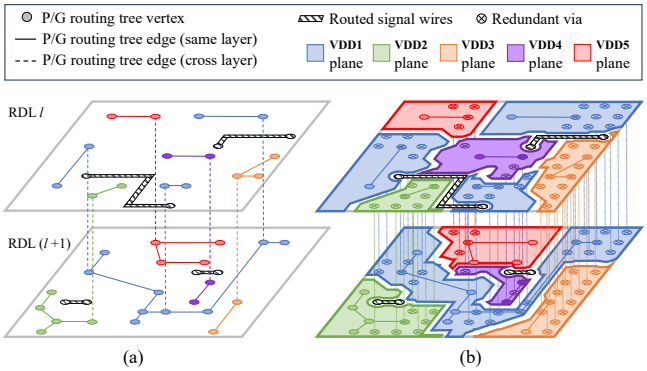

Figure 1: Simultaneous power and signal routing on RDLs and power plane generation. (a) An RDL routing instance, including the routing trees of five power nets, VDD1, VDD2, VDD3, VDD4, and VDD5, and routed signal wires, on two adjacent RDLs. (b) The optimal power planes of each power net resulting from the power routing trees.

Fig. 1 illustrates a 3D-IC RDL routing instance containing both power and signal nets. First of all, the global routing for both power and signal nets must be simultaneously performed in order to ensure the routability of not only signal nets but also power nets. After obtaining the global routing trees, the detailed routing for all signal nets is then executed, followed by power plane generation for the rest power nets. Fig. 1(a) shows the global routing trees of all power nets resulting from simultaneous global routing for both power and signal nets. The routed wires and vias after detailed routing for all signal nets are also highlighted in Fig. 1(a). Based on the global routing trees of all power nets and the obstacles of signal wires and vias, Fig. 1(b) further demonstrates the optimal power planes, which maximizes both the space utilization of each RDL and the number of redundant vias between the power planes of the same power net on adjacent RDLs, leading to better power integrity.

1.1 Related Works

Although the RDL routing and pin assignment problems has been extensively studied in the literature [7–10], all these works only focused on interconnection optimization for signal nets. None of them discussed about power delivery issues with an increasing number of power supplies and power domains in modern 3D ICs. Consequently, creating power planes on RDLs is still a manual and time-consuming task, and the automatic power

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
DAC '24, June 23–27, 2024, San Francisco, CA, USA
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0601-1/24/06
https://doi.org/10.1145/3649329.3657315

plane generation and optimization for multiple power nets on RDLs in 3D ICs remains an open problem.

Other recent works [11, 12] focused on PCB power network layout synthesis. Bairamkulov *et al.* [11] presented a power routing tool based on a graph-search algorithm. They further apply the simulated annealing (SA) algorithm to reduce the impedance of the power network by dilation and erosion operations. Instead of applying the graph-search algoirthm and SA, Liao *et al.* [12] proposed to use the genetic optimizer and multi-layer perception (GOMLP) for power plane generation on PCBs. Their objective is to minimize the total number of power plane islands in order to ensure the connectivity of each power net.

We observe some disadvantages of the previous works.

- Most of the problem formulations in the previous works routes power nets and signal nets separately. Such formulation may cause routability issues of either power nets or signal nets. Although some signal routing methods could be applied to power net routing, the routing styles are quite different.
- Power integrity is crucial for each power distribution network. The previous works mainly focus on reducing the resistance or impedance of each power plane on one single layer. They do not consider cross-layer power integrity optimization among the power planes of each power net on any two adjacent layers.
- Last but not least, different from PCB design, the sources (fan-in pads/bumps) and sinks (fan-out pads/bumps) of each power net in a 3D IC are usually on the opposite sides of the interposer. The power planes of a power net may be distributed on all RDLs, and different power nets may compete for the routing resources on each RDL. Consequently, the problem of power plane generation on RDLs in 3D ICs is even more difficult than that on PCBs.

1.2 Our Contributions

The significant contributions of this paper can be summarized in the following:

- To the best of our knowledge, this is the *first* work in the literature that addresses the power plane generation problem in modern 3D ICs, which is even more sophisticated than that in conventional PCBs, considering the routing resource competition among different power nets on each RDL and cross-layer power integrity of each power net.
- Different from the previous works which only focused on routing either signal nets or power nets, we present the *first* problem formulation in the literature for multiple power plane generation in 3D ICs that guarantees the routability of both signal nets and power nets.
- Based on the presented problem formulation, we introduce the *first* Voronoi diagram-based power plane generation method to achieve balanced power plane distribution on each RDL according to the global routing trees of all power nets, while maximizing space utilization under design rule constraints.
- We further refine the power planes on each RDL resulting from the Voronoi diagram by considering signal wires and vias as obstacles, and optimize the cross-layer power planes with more redundant vias for better power integrity.
- Based on the industry benchmarks with up to fifteen power nets on four RDLs in 3D ICs, our experimental results show that the power planes generated by the proposed method can achieve significant improvement on cross-layer power integrity in terms of the total number of redundant vias, compared with the manual approach and the latest work for PCBs.

The rest of this paper is organized as follows: Section 2 presents the problem formulation. Sections 3–6 detail the proposed flow and algorithms.

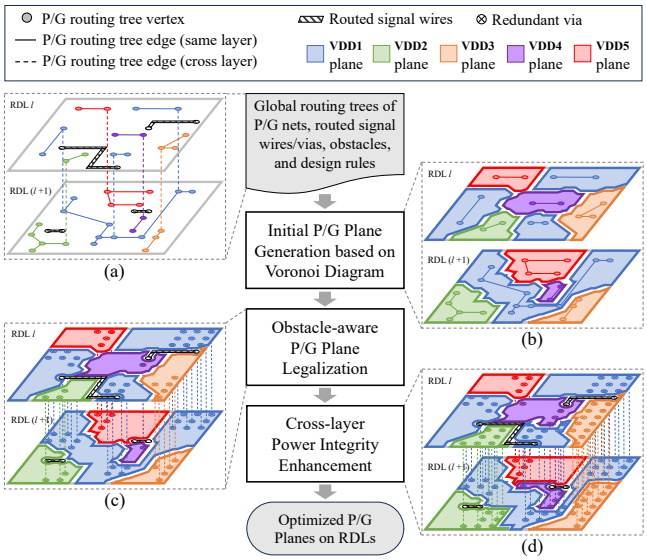

Figure 2: The proposed algorithm flow, and the corresponding examples at each stage. (a) The problem input, including the power routing trees and the obstacles. (b) The initial power planes of different power nets resulting from the power routing trees while ignoring obstacles. (c) The refined power planes with the consideration of obstacles, where the number of redundant vias is not optimal. (d) The optimal power planes which maximizes cross-layer redundant vias for power integrity enhancement.

Section 7 reports the experimental results, and finally section 8 concludes this work.

2 PROBLEM FORMULATION

The *obstacle-avoiding multiple redistribution layer routing problem* has been well formulated and solved in [8]: Given a set of RDLs, a set of I/O pads, a set of bump pads, a set of obstacles, a set of unified-assignment netlists, and RDL design rules, connect all the netlists such that the routability is maximized, the total wirelength is minimized, and all design rules are satisfied. Although such formulation can also simultaneously route the power nets in addition to the signal nets, it does not generate power planes for the routed power nets on RDLs. Therefore, we shall further define the RDL power plane generation problem as follows:

Problem 1. ***Obstacle-Avoiding Multiple Redistribution Layer Power Plane Generation****: Given a set of RDLs, a set of obstacles including routed signal wires and vias, RDL design rules, and a set of global routing trees of all power nets, create all the power/ground planes such that the area utilization is maximized, the cross-layer power integrity (i.e. total via numbers) is maximized, and all design rules are satisfied.*

Based on the problem definition and formulation, Fig. 1(a) gives an example of the problem input, while Fig. 1(b) shows the expected result.

3 ALGORITHM OVERVIEW

In order to solve Problem 1, which is the first time defined in the literature, we propose a novel algorithm flow, as demonstrated in Fig. 2. Our algorithm flow consists of three major stages:

- **Initial Power/Ground Plane Generation based on Voronoi Diagram** constructs a Voronoi diagram for each RDL according to the global routing tree vertices and edges of the power nets on the same RDL. Based on the Voronoi diagram, each RDL can be dissected into different power planes corresponding to the global routing trees of

those power nets, as shown in Fig. 2(b), while achieving the highest area utilization.

- **Obstacle-aware Power/Ground Plane Legalization** realizes power planes by considering design rules and trims off the areas of the routed signal wires/vias and other obstacles from the initial power planes based on the Boolean operations. It further refines the power planes by identifying and reconnecting the floating planes while maintaining the area utilization, as shown in Fig. 2(c).
- **Cross-layer Power Integrity Enhancement** further enlarges the overlapped power plane areas on adjacent RDLs belonging to the same power net by converting some fractions of power planes into different power nets for more redundant via insertion, as shown in Fig. 2(d). Consequently, both area utilization and cross-layer power integrity are maximized.

The following sections will further explain the algorithms of each stage in greater detail.

4 INITIAL POWER PLANE GENERATION

Instead of directly solving Problem 1, we ignore all obstacles at the beginning to reduce the complexity of the problem. We define the simplified initial power plane generation problem as follows:

Problem 2. ***Initial Power Plane Generation****: Given a set of RDLs, design rules, and the global routing trees of all power nets, create all the power/ground planes such that the area utilization is maximized, while all design rules are satisfied.*

We employ the idea of constructing the Voronoi diagram to solve Problem 2 because of its advantages of geometric properties and efficiency. Given a set of *seeds* which are randomly distributed on a two-dimensional (2D) surface, a Voronoi diagram partitions the 2D surface into a set of connected convex polygons, or *Voronoi cells*, where each Voronoi cell, c_i, contains a seed, s_i, and all points in c_i are closer to s_i than any other seeds. All Voronoi cells form a tessellation which completely covers the 2D surface with full area utilization. By applying the Fortune's algorithm [13], it takes only $O(n \lg n)$ to construct a Voronoi diagram, where n denotes the number of seeds.

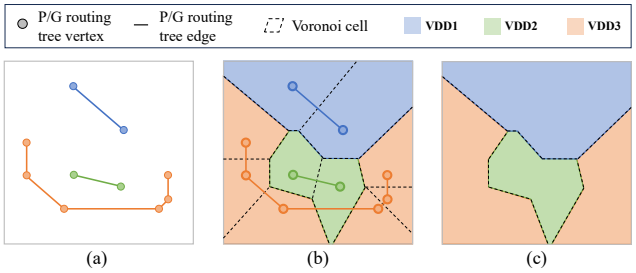

Figure 3: An example of constructing power planes on an RDL based on the power routing trees and Voronoi diagram. (a) The global routing trees of three power nets on the RDL. (b) The Voronoi diagram with the seeds corresponding to the global routing tree vertices in (a). (c) The resulting power planes of the three power nets on the RDL, where VDD3 has disconnected power plane violation.

We are the first in the literature which propose to generate the power planes on RDLs by taking the aforementioned advantages of the Voronoi diagram. We construct the Voronoi diagram on each RDL by considering at least the global routing tree vertices of all power nets on that RDL as the initial seeds, which is demonstrated Fig. 3. Fig. 3(b) shows the resulting Voronoi diagram with the seeds corresponding to the global routing tree vertices in Fig. 3(a). Each Voronoi cell in Fig. 3(b) represents a power plane of the corresponding power net. If two adjacent Voronoi cells belong to the same power net, the power planes can be merged, as shown in Fig. 3(c). In this example, both VDD1 and VDD2 result in a single connected power plane after merging their Voronoi cells, respectively. However, VDD3 in Fig. 3(c) results in two separate power planes, which violates the original power routing tree of VDD3 in Fig. 3(a).

Definition 1. ***Disconnected Power Plane Violation****: Given a globally routed power net on an RDL with the power routing tree, the power plane resulting from the power routing tree must be connected. Otherwise, the disconnected power plane violation occurs.*

To avoid the disconnected power plane violation when generating power planes based on the Voronoi diagram, all Voronoi cells belonging to the same power net must be connected. In order to ensure a Voronoi diagram without any disconnected power plane violation, we have the following lemma and theorem while omitting the proofs due to the page limit.

Lemma 1. *When constructing a Voronoi diagram with the seeds corresponding to the power routing tree vertices, the disconnected power plane violation occurs if any circumscribed circle, which is formed by a power routing tree edge of the power net, n_i, as its circumdiameter, contains any power routing tree vertex of the other power net, n_j.*

Theorem 1. *The power planes of each RDL resulting from the Voronoi diagram are valid without any disconnected power plane violation if the smallest circumscribed circle of each power routing tree edge does not contain any other vertex.*

Based on Lemma 1 and Theorem 1, we shall insert extra vertices/seeds to separate those power routing edges whose smallest circumscribed circle contains other vertices. Once those long power routing edges are separated into shorter ones whose smallest circumscribed circles do not contain any other vertex, the power planes resulting from the Voronoi diagram with seeds based on both the original power routing tree vertices and the inserted vertices are valid without any disconnected power plane violation.

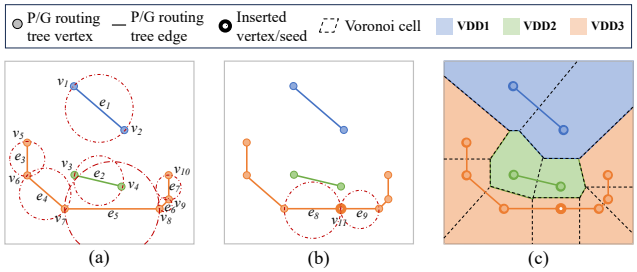

Figure 4: Vertex/Seed insertion to avoid disconnected power plane violation. (a) The smallest circumscribed circles of all power routing tree edges. (b) The newly inserted vertex/seed, v_{11}, which avoids disconnected power plane violation. (c) The resulting Voronoi diagram and the power planes of all power nets without disconnected power plane violation.

Fig. 4 gives an example of vertex/seed insertion to avoid disconnected power plane violation. Fig. 4(a) highlights the smallest circumscribed circles of the power routing tree edges, e_1, e_2, ..., e_7. The smallest circumscribed circle means that the circumdiameter of each circle is the corresponding edge length. It should be noted the circumscribed circle of e_5 contains the vertex, v_4. Therefore, we shall insert an extra vertex/seed, v_{11}, by projecting v_4 to e_5. After inserting v_{11}, e_5 is separated into two shorter edges e_8 and e_9, as shown in Fig. 4(b). We shall further verify whether the newly inserted vertex is contained by any other circumscribed circle and whether the smallest circumscribed circles of the new edges contain other vertices. The vertex/seed insertion will stop until no circumscribed circle

contains other vertices. Fig. 4(c) shows the resulting Voronoi diagram and the power planes of all power nets without disconnected power plane violation after vertex/seed insertion.

5 OBSTACLE-AWARE POWER PLANE LEGALIZATION

After generating the initial power/ground planes based on the Voronoi diagram, we shall realize the power/ground planes according to the design rules while considering the routed signal wires/vias and other obstacles. The design rules include both the angle rule and the spacing rule. For the angle rule, the boundary lines of power planes are restricted to 0, 45, 90, and 135 degrees. To convert any angle boundary lines into the restricted degrees of angles, we rotate each boundary line with the axis at its center point to the closest satisfied degree. To satisfy the spacing rule, we apply the Boolean operations on polygons. We first upsize the polygon of each power plane based on the spacing rule and then remove the overlap area. Fig. 5(a) shows the initial power planes of the three power nets resulting from the Voronoi diagram in Fig. 4(c), and Fig. 5(b) realizes the power planes according to the angle and spacing rules. Other design rules, such as the acute angle, will be handled at the post-processing stage, which is beyond the scope of this work.

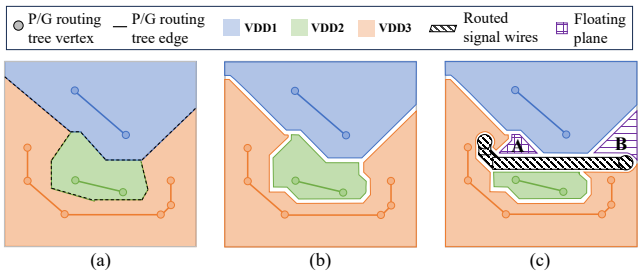

Figure 5: Plane realization and obstacle consideration. (a) The power planes resulting from the Voronoi diagram. (b) Power plane realization according to design rules. (c) Power plane realization considering obstacles, which may lead to floating planes.

Once the power planes are realized, we shall further consider the routed signal wires/vias and other obstacles. We apply the Boolean operator, *NOT*, to trim off the up-sized signal wires/vias and obstacles due to the spacing rule from the realized power planes, as shown in Fig. 5(c). It should be noted that such a Boolean operation may lead to some floating planes.

Definition 2. ***Floating Plane****: a floating plane is a dangled plane that does not associate with any global routing tree vertex of a power net after trimming off the obstacles from the initial power planes.*

In Fig. 5(c), the regions, *A* and *B*, become floating planes after trimming off the area of signal wires/vias because they are dangled and do not associate with the power net, VDD3, anymore.

Instead of removing the floating planes, we shall reconnect the floating planes to their adjacent power planes of different power nets to achieve even better area utilization.

Problem 3. ***Floating Plane Reconnection****: Given a set of power planes and a set of floating planes, reconnect each floating plane to one of its adjacent power planes such that the area utilization is maximized.*

Different from the previous works [14–18] which reconnect different polygons of the same net by searching for the shortest path, Problem 3 reconnect the polygon of each floating plane to any power net by search for the longest common borders between the floating plane and its adjacent power planes in order to maximize the area utilization. The common borders can be found by upsizing the floating planes and the neighboring non-floating power planes.

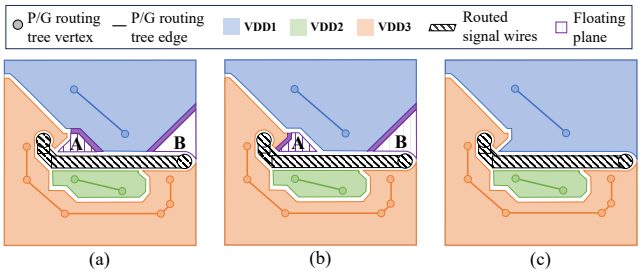

Figure 6: Floating plane reconnection. (a) Both floating planes, *A* and *B*, can be reconnected to the power plane of VDD1. (b) The floating planes, *A* and *B*, can be reconnected to the power planes of VDD3 and VDD1, respectively. (c) The refined power planes after floating plane reconnection.

Fig. 6 gives an example of floating plane reconnection. When sizing up the floating planes, *A* and *B*, and their neighboring non-floating power planes, the floating plane, *B*, has only one common border with the power plane of VDD1, and the floating plane, *A*, has common borders with the power planes of both VDD1 and VDD3. Since the border between *A* and VDD1 is longer than that between *A* and VDD3, reconnecting *A* to VDD1 results in higher area utilization.

6 CROSS-LAYER POWER INTEGRITY ENHANCEMENT

After solving Problems 2 and 3, the result is already a legal solution, which maximizes power plane area utilization while satisfying the design rule constraints with the consideration of all obstacles. However, Problems 2 and 3 only focus on every single RDL without considering cross-layer power plane co-optimization. The cross-layer power integrity is essential especially when the fan-in pad and fan-out bumps of a power net are on different sides of the interposer. We shall further maximize the number of vias between the power planes on adjacent RDLs by enlarging the overlapping area. As the RDLs have been fully occupied by power planes resulting from the former stages, all we can do is trade some fractions of power planes among different power nets. Fig. 7 illustrates the improvement of the power plane via numbers after cross-layer power integrity enhancement between two adjacent RDLs.

Problem 4. ***Power Plane Trading for Cross-Layer Power Integrity Enhancement****: Given the legalized non-floating power planes on different RDLs with maximized utilization, trade some fractions of power planes among different power nets such that the overlapping ratio of the power planes of the same power net on adjacent RDLs and the total number of power plane vias are maximized without sacrificing power plane area utilization.*

Before solving Problem 4, we give the following definitions:

Definition 3. ***Stacked Plane****: A power plane is a stacked plane if it overlaps with two or more planes belonging to the same net on consecutive adjacent RDLs.*

Definition 4. ***Hard Plane****: A power plane is a hard plane if it is not a stacked plane, while overlaps with another power plane belonging to the same net on the adjacent RDL.*

Definition 5. ***Soft Plane****: A power plane is a soft plane if it is neither a stacked plane nor a hard plane.*

Definition 6. ***Tradeable Plane Set****: A set of two overlapped power planes on adjacent RDLs is tradeable if one of them is a soft plane, and the other is either a soft plane or a stacked plane.*

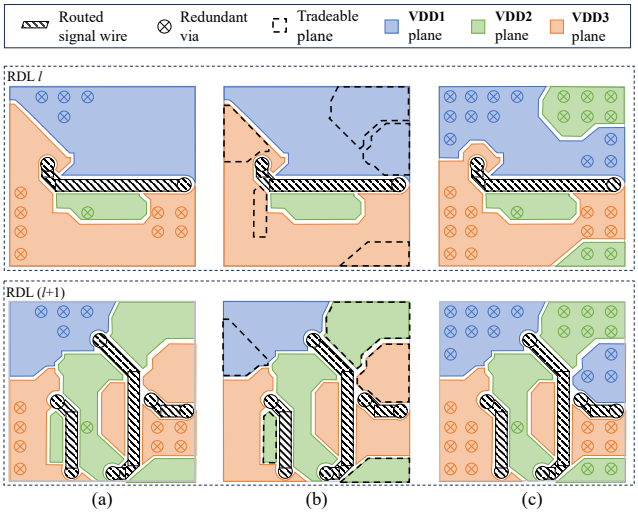

Figure 7: Cross-layer power integrity enhancement between two adjacent RDLs. (a) The power planes resulting from Section 5 with only a few power plane vias between RDL_l and RDL_{l+1}. (b) The tradeable power plane fractions for enhancing power integrity. (c) The resulting power planes with much more power plane vias after power plane trading.

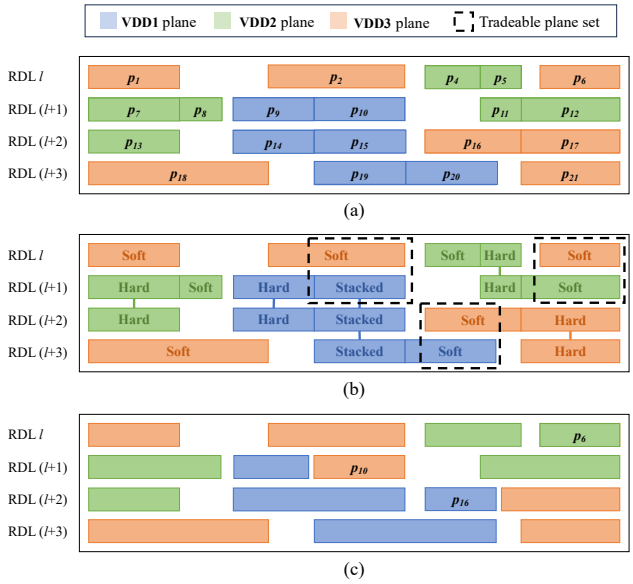

Figure 8: The cross-section view of power plane trading for cross-layer power integrity enhancement. (a) The fragmented power planes on four RDLs before power plane trading. (b) The tradeable plane set. (c) The resulting power planes after power plane trading, where VDD1 trades p_{16} from VDD3, VDD2 trades p_4 from VDD3, and VDD3 trade p_{10} from VDD1.

To solve Problem 4, first of all, we shall fragment the power planes on each RDL according to the power plane boundaries on the adjacent RDL. Once the power planes are fragmented into smaller pieces, we label each fragmented power plane with either a stacked plane, a hard plane, or a soft plane. With the label of each plane, we shall collect all tradeable plane sets, according to Definition 6 between any two adjacent RDLs. For the overlapped power planes, p_i and p_j, in each tradeable plane set, they must belong to different power nets, n_a and n_b, respectively. We can either trade p_i from n_a to n_b, or trade p_j from n_b to n_a. The objective of the trading is to maximize the overlap ratio of all power planes belonging to the same power net on adjacent RDLs, which can be calculated by Equation (1), where A_{P_l} is total area of the power planes on the RDL_l.

$$Overlap\ Ratio = \frac{A_{P_l} \bigcap A_{P_{l+1}}}{A_{P_l} \bigcup A_{P_{l+1}}} \tag{1}$$

A larger overlap ratio between the power planes on adjacent RDL belonging to the same power net indicates that more redundant vias can be inserted leading to higher power integrity. We apply the greedy algorithm to achieve the optimal power plane trading for all tradeable plane sets.

Fig. 8 demonstrates an example of power plane trading for cross-layer power integrity enhancement with a cross-section view. In Fig. 8(a), the power planes are fragmented into smaller pieces according to the power plane boundaries on the adjacent RDL. Fig. 8(b) shows the label of each fragmented power plane, as well as the tradeable plane sets. After applying the greedy algorithm to maximize the total overlap ratio of power planes belonging the the same net on adjacent RDLs, the optimal power plane result is shown in Fig. 8(c).

7 EXPERIMENTAL RESULTS

We implemented the proposed power plane generation algorithm in the C++ programming language with Boost Geometry 1.8.0 for the polygon computation. Our algorithm was executed on an Intel i9 3.4GHz Linux workstation with 64GB memory, and tested based on the industry benchmarks, as shown in Table 1. The benchmarks include five cases with the number of power nets ranging from 3 to 14 and the number of signal nets ranging from 27 to 250. Each power net is globally routed (not physically routed yet) with a 3D routing tree on four RDLs connecting a large number of fan-in pads and fan-out bumps with hundreds of tree vertices. Each benchmark also has a globally routed (not physically routed yet) ground net connecting even more fan-in pads and fan-out bumps. All signal nets in each benchmark had been physically routed wires and vias, which are obstacles during power plane generation.

To demonstrate the effectiveness and efficiency of our algorithm, we experimentally compared our algorithm with the most recent power plane generation method [12], namely GOMLP, and the manual approach from the industry. Tables 2 and 3 show the experimental results. Compared with GOMLP [12], our approach results in similar power plane area utilization, but achieves 2.5X via number between cross-layer power planes, 16% more ground plane area, and 29% more ground plane vias. Compared with the manual approach from the industry, although our approach results in 28% less ground plane area and 40% less ground plane vias, our approach leads to 9X power plane area and 100X via number between cross-layer power planes. Most importantly, the runtime based on our approach can be significantly reduced from hours or even days to less than one minute according to Table 3.

It should also be noted that, according to Table 1, the total routing tree wirelength of the power nets in Cases 0–2 is much longer than the routing tree wirelength of the ground net. Consequently, the percentages of power plane area and ground plane area resulting from GOMLP [12] and our approach are more balanced than those resulting from the manual design.

In summary, the reasons why our proposed Voronoi diagram-based multiple power plane generation algorithm can achieve better performance in power plane area utilization, cross-layer power plane vias, and runtime are analyzed as follows:

- We construct the Voronoi diagram based on the global routing trees of both power and ground nets on each RDL, which not only maximizes the power plane area utilization but also balances the power plane area across all RDLs according to the distributions of the power routing trees.

Table 1: Benchmarks statistics including the number of layers ($|L|$), the number of power nets ($|N_P|$), the number of power fan-in pads ($|Q_P|$), the number of power fan-out bumps ($|B_P|$), the number of power routing tree vertices ($|V_P|$), the total wirelength of power routing trees ($|WL_P|$), the number of ground nets ($|N_G|$), the number of ground fan-in pads ($|Q_G|$), the number of ground fan-out bumps ($|B_G|$), the number of ground routing tree vertices ($|V_G|$), the total wirelength of ground routing trees ($|WL_G|$), and the number of signal nets (N_S).

Benchmarks	$\|L\|$	$\|N_P\|$	$\|Q_P\|$	$\|B_P\|$	$\|V_P\|$	$\|WL_P\|(\mu m)$	$\|N_G\|$	$\|Q_G\|$	$\|B_G\|$	$\|V_G\|$	$\|WL_G\|(\mu m)$	$\|N_S\|$
Case 0	4	3	42	48	301	8467.46	1	113	81	704	2090.80	27
Case 1	4	7	507	94	580	55184.64	1	557	142	1364	13827.32	62
Case 2	4	6	207	77	412	35130.22	1	565	142	1388	14075.32	62
Case 3	4	13	28	13	105	11216.81	1	1846	445	3260	41482.05	250
Case 4	4	14	48	14	136	21609.04	1	1846	445	3260	41482.05	250

Table 2: Comparisons of the percentage of total power plane area ($|A_P|$), the number of vias in the power planes ($|I_P|$), the percentage of total ground plane area ($|A_G|$), the number of vias in the ground planes ($|I_G|$) for GOMLP [12], the manual approach from industry, and Ours.

Benchmarks	GOMLP[12]				Manual				Ours w/o Cross-Layer PI Opt.				Ours w/ Cross-Layer PI Opt.			
	$\|A_P\|$	$\|I_P\|$	$\|A_G\|$	$\|I_G\|$	$\|A_P\|$	$\|I_P\|$	$\|A_G\|$	$\|I_G\|$	$\|A_P\|$	$\|I_P\|$	$\|A_G\|$	$\|I_G\|$	$\|A_P\|$	$\|I_P\|$	$\|A_G\|$	$\|I_G\|$
Case 0	30.71%	6482	67.25%	30703	2.39%	175	97.18%	60868	33.34%	11150	66.66%	33200	38.52%	17274	61.48%	32241
Case 1	40.92%	6905	46.62%	16751	8.10%	211	91.60%	43416	33.54%	7985	66.47%	27704	36.82%	10935	63.21%	26381
Case 2	22.10%	3863	63.74%	26493	5.73%	146	94.26%	48019	21.17%	4998	78.84%	37183	24.51%	8030	75.51%	36109
Case 3	5.99%	1583	79.07%	102070	0.05%	40	99.75%	150899	4.04%	3023	95.76%	145470	4.34%	3840	95.46%	145322
Case 4	7.84%	861	60.17%	71817	0.07%	58	99.73%	149964	6.74%	5916	92.91%	137977	7.87%	8747	91.78%	137119
Comp.	1.04	0.40	0.84	0.71	0.11	0.01	1.28	1.40	0.89	0.69	1.04	1.02	1.00	1.00	1.00	1.00

- We maximize the overlapping ratio of power plane areas on adjacent RDLs that belong to the same power net for more redundant vias, which helps ensure cross-layer power integrity.
- Our power plane generation and optimization algorithms can be done in polynomial time, which is extremely efficient compared to both manual and non-deterministic approaches.

Table 3: Comparisons of runtime for GOMLP [12], the manual approach from industry, and Ours.

Benchmarks	GOMLP [12]	Manual	Ours w/o Cross-Layer PI Opt.	Ours w/ Cross-Layer PI Opt.
Case 0			0.59 s	6.15 s
Case 1	minutes	hours	3.99 s	12.92 s
Case 2	to	to	2.87 s	10.19 s
Case 3	hours	days	18.18 s	32.68 s
Case 4			13.18 s	35.18 s

8 CONCLUSIONS

In this paper, we have presented the new problem formulation for multiple power plane generation on RDLs in 3D ICs that guarantees the routability of both signal nets and power nets. We have proposed to generate the initial power planes of all power nets based on the Voronoi diagram and their global routing trees while maximizing routing space utilization under design rule constraints. We have further proposed to refine the power planes by considering routed signal wires and vias as obstacles. We have finally enhanced the cross-layer power integrity. Our experimental results have shown that the power planes generated by our method can achieve significant improvement in cross-layer power integrity, compared with the manual approach and the recent work.

REFERENCES

[1] C.-F. Tseng, C.-S. Liu, C.-H. Wu, and D. Yu, "InFO (wafer level integrated fan-out) technology," in *Proc. ECTC*, 2016, pp. 1–6.

[2] Y.-H. Lin, M. Yew, S. Chen, M. Liu, P. Kavle, T. Lai, C. Yu, F. Hsu, C. Chen, T. Fang, C. Hsu, K. Lee, C. Lin, P. Lin, and S.-P. Jeng, "Multilayer RDL interposer for heterogeneous device and module integration," in *Proc. ECTC*, 2019, pp. 931–936.

[3] H. Chen, C.-K. Cheng, A. Kahng, Q. Wang, and M. Mori, "Optimal planning for mesh-based power distribution," in *Proc. ASP-DAC*, 2004, pp. 444–449.

[4] S.-H. Wang, G.-H. Liou, Y.-Y. Su, and M. P.-H. Lin, "IR-aware power net routing for multi-voltage mixed-signal design," in *Proc. DATE*, 2019, pp. 72–77.

[5] M.-Y. Huang, H.-M. Chen, K.-N. Chen, S.-H. Wu, Y.-M. Lee, and A.-Y. Su, "A design flow for micro bump and stripe planning on modern chip-package co-design," in *Proc. ECTC*, 2020, pp. 2236–2241.

[6] D. Hyun, W. Lee, J. Park, and Y. Shin, "Integrated power distribution network synthesis for mixed macro blocks and standard cells," *IEEE TCAS-II*, vol. 70, no. 6, pp. 2211–2215, 2023.

[7] H.-T. Wen, Y.-J. Cai, Y. Hsu, and Y.-W. Chang, "Via-based redistribution layer routing for InFO packages with irregular pad structures," *IEEE TCAD*, vol. 41, no. 12, pp. 5554–5567, 2022.

[8] Y.-T. Chen and Y.-W. Chang, "Obstacle-avoiding multiple redistribution layer routing with irregular structures," in *Proc. ICCAD*, 2022, pp. 1–6.

[9] Y.-H. Chang, H.-T. Wen, and Y.-W. Chang, "Obstacle-aware group-based length-matching routing for pre-assignment area-I/O flip-chip designs," in *Proc. ICCAD*, 2019, pp. 1–8.

[10] Y.-K. Ho, H.-C. Lee, W. Lee, Y.-W. Chang, C.-F. Chang, I.-J. Lin, and C.-F. Shen, "Obstacle-avoiding free-assignment routing for flip-chip designs," *IEEE TCAD*, vol. 33, no. 2, pp. 224–236, 2014.

[11] R. Bairamkulov, A. Roy, M. Nagarajan, V. Srinivas, and E. G. Friedman, "SPROUT—Smart power routing tool for board-level exploration and prototyping," *IEEE TCAD*, vol. 41, no. 7, pp. 2263–2275, 2022.

[12] H. Liao, V. Patil, X. Dong, D. Shanbhag, E. Fallon, T. Hogan, M. Spasojevic, and L. B. Kara, "Hierarchical automatic power plane generation with genetic optimization and multilayer perceptron," arXiv:2210.16314 [cs.NE], 2022.

[13] S. Fortune, "A sweepline algorithm for Voronoi diagrams," in *Proceedings of the 2nd Annual Symposium on Computational Geometry*, 1986, p. 313–322.

[14] K.-S. Hu, M.-J. Yang, Y.-H. Huang, B.-Y. Wong, and C. Shen, "ICCAD-2017 CAD contest in net open location finder with obstacles," in *Proc. ICCAD.* , 2017, p. 863–866.

[15] G.-Q. Fang, Y. Zhong, Y.-H. Cheng, and S.-Y. Fang, "Obstacle-avoiding open-net connector with precise shortest distance estimation," *IEEE TCAD*, vol. 39, no. 5, pp. 1096–1108, 2020.

[16] Y.-Y. Su, S.-H. Wang, W.-L. Wu, and M. P.-H. Lin, "Corner-stitching-based multilayer obstacle-avoiding component-to-component rectilinear minimum spanning tree construction," *IEEE TCAD*, vol. 39, no. 3, pp. 675–685, 2020.

[17] B.-H. Jiang and H.-M. Chen, "Extending ML-OARSMT to net open locator with efficient and effective boolean operations," in *Proc. ICCAD*, 2018, pp. 1–8.

[18] R.-Y. Wang, C.-C. Pai, J.-J. Wang, H.-T. Wen, Y.-C. Pai, Y.-W. Chang, J. C.M. Li, and J.-H. Jiang, "Efficient multi-layer obstacle-avoiding region-to-region rectilinear steiner tree construction," in *Proc. DAC*, 2018, pp. 1–6.